ENTERPRISE SYSTEMS INTEGRATION

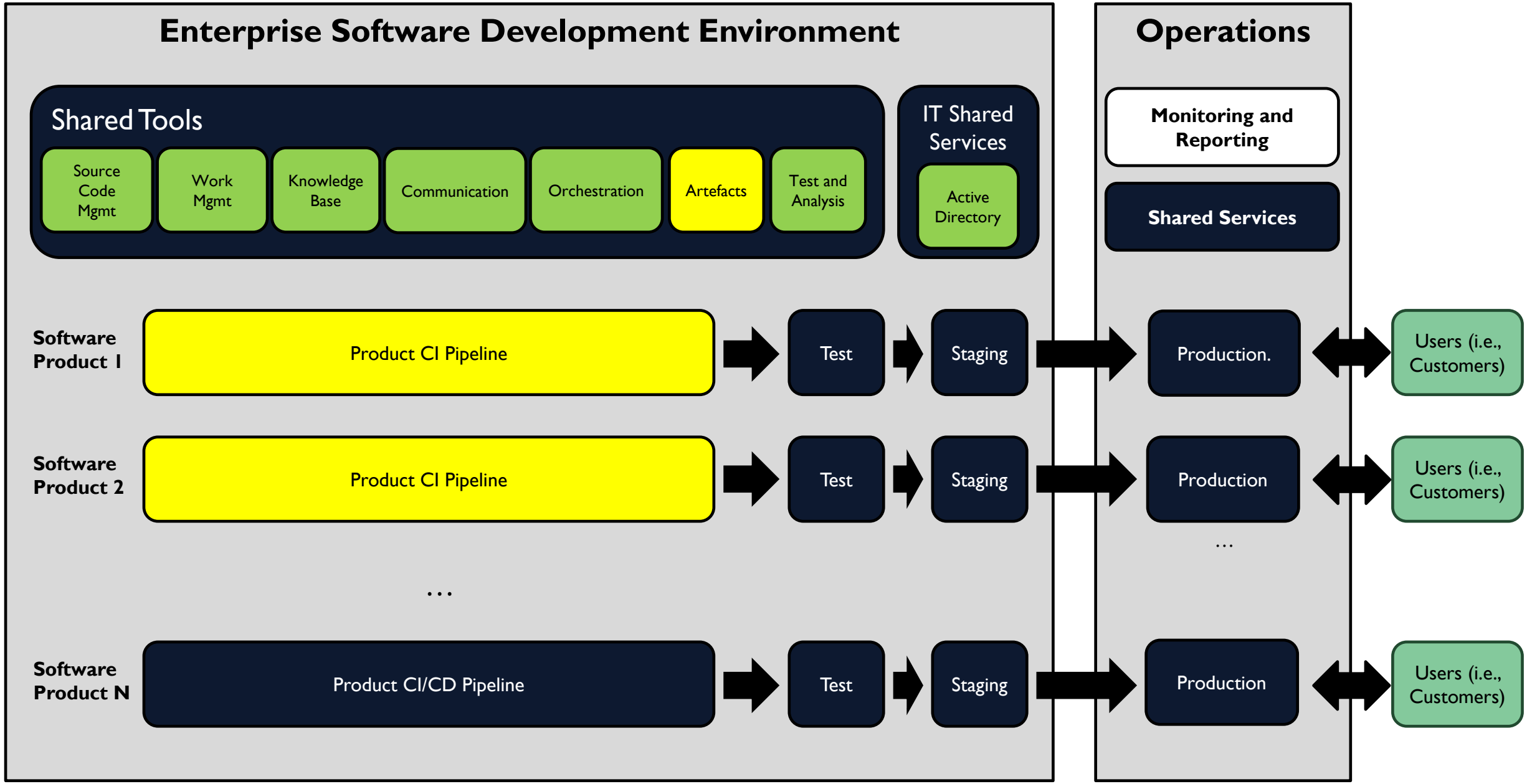
ACIT4850 – WINTER 2026

AGENDA – LESSON 10

- Quiz 9 on D2L
- What's Left?
  - Labs, Assignments and Final
- Topics
  - CI/CD Pipelines
  - Containerization – Docker
- Lab 10 – Containerization

QUIZ 9

- Open Book – You should have the Lesson 10 reading materials available
- You have 15 minutes to complete
- We will resume in 15 minutes, or when everyone is done


Enterprise Software Development Environment
Shared Tools
Source Code Mgmt
Work Mgmt
Knowledge Base
Communication
Orchestration
Artefacts
Test and Analysis
IT Shared Services
Active Directory
Software Product 1
Product CI Pipeline
Test
Staging
Software Product 2
Product CI Pipeline
Test
Staging
...
Software Product N
Product CI/CD Pipeline
Test
Staging
Operations
Monitoring and Reporting
Shared Services
Production.
Production
...
Production
Users (i.e., Customers)
Users (i.e., Customers)
Users (i.e., Customers)

THE ROADMAP (AKA COURSE SCHEDULE)

Week	Topics	Notes
1	• Components of an Enterprise Development Environment • Software Source Code Management	Lab 1
2	• Work Management and Knowledge Base Tools	Lab 2, Quiz 1
3	• Tool Selection – Requirements • Integration and Security	Lab 3, Quiz 2
4	• Tool Selection – Stakeholders/Process • Continuous Integration (CI) Tool • CI Tool Setup	Lab 4, Quiz 3
5	• CI Pipelines – Python	Lab 5, Quiz 4,Assignment 1 Due
6	• CI Pipelines – Shared Libraries	Lab 6, Quiz 5
7	• CI Pipelines – Java and Static Code Analysis	Lab 7, Quiz 6
8	**Midterm Exam**	
9	• CI Pipelines – Alternate Tools	Lab 8, Quiz 7
10	• CI Pipelines – Artifact Management (Java)	Lab 9, Quiz 8, Assignment 2 Due
11	**Computing Reading Week**	
12	• Continuous Delivery (CD) • CD Pipelines - Containerization	Lab 10, Quiz 9
13	• CD Pipelines – Deployment 1 • Developer Workflows with Jenkins	Lab 11, Quiz 10
14	• CD Pipelines – Deployment 2 • Developer Workflows with GitLab CI	Lab 12, Quiz 11, Assignment 3 Due
15	• Final Exam Preview	Lab 12/Assignment 3 Demos in Class
16	**Final Exam**	

LABS – LAST LABS LEFT

Lab 10 – This Week

- Add a Containerization Stage to your build pipelines
- Due end of next class (demo)

Lab 11 - Next Class

- Simple Deployment of Docker Images
- Exercise a Merge Request

Lab 12

- Repeat the above on GitLab CI

ASSIGNMENTS AND FINAL EXAM

Assignment 3

- CI/CD Pipeline using a Shared Library
- Deployment to a separate VM
- Posted this week
- Due for demo on or before last class

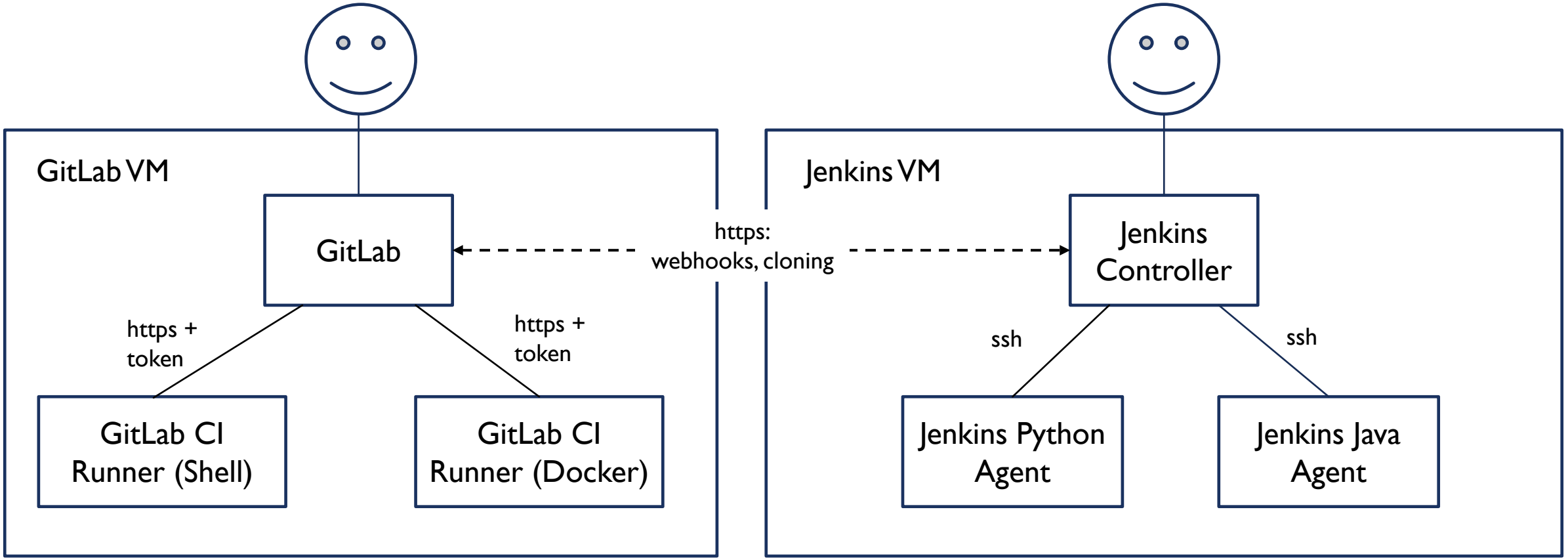
Final Exam

- In Person – Week of April 20th
- "Written" and "Practical" components

ASSIGNMENT 2

- Assignment 2
  - How would you compare Jenkins and GitLab CI?
- Compare our deployments of Jenkins vs GitLab CI (next slide)

JENKINS VS GITLAB CI



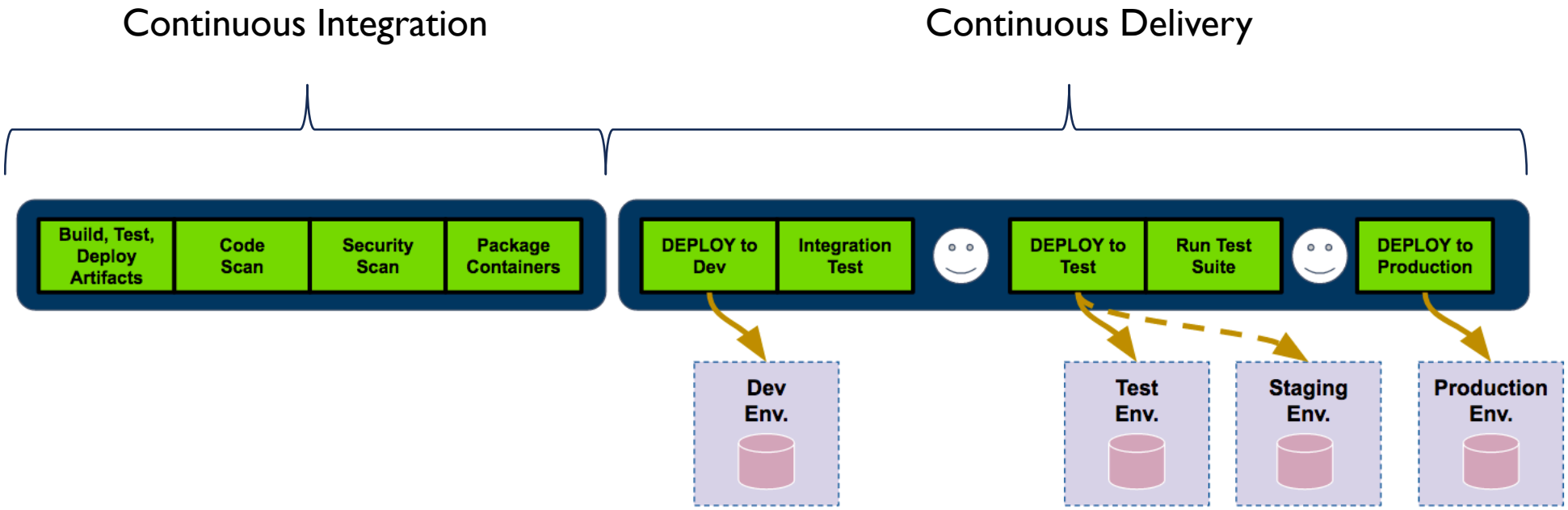
How might we change this for a Production installation?

CI/CD DEFINITIONS

- Continuous Integration – Includes the following:
  - Build
  - Unit level tests (those that do not require a deployment)
  - Static code analysis
  - Packaging (i.e., zip or container)
  - Push to an Artifact Repository
- Continuous Delivery
  - Getting code into a deployable state (you can argue that Packaging/Artifact Management is part of CD rather than CI)
  - Automation of the deployment of that deployable code into internal environments
    - Development, Test, Staging, Production

CI/CD PIPELINE

To enable Continuous Delivery, we need a pipeline. This is effectively the pipeline we have been creating in this course.



TARGET ENVIRONMENT

- **Development** – Environment where developers can integrate and test their code
  - Ideally they can do this on their own workstation
  - If not, there needs to be a separate environment
- **Test** – Environment where developers and testers can validate and verify the software
- **Staging** – Environment as close to production as possible. Where acceptance testing and final smoke and deployment tests occur.
- **Production** – Environment where the "live" software is deployed

CONTAINERIZATION

- Containerization is a lightweight alternative to full machine virtualization that involves encapsulating an application in a container with its own operating environment.
- This provides many of the benefits of loading an application onto a virtual machine, as the application can be run on any suitable physical machine without any worries about dependencies.
- The container includes all the dependencies needed to run the application. **Makes it easy to deploy in a consistent manner.**
- Docker is an open-source software product that allows us to define, build and run containers.

DOCKER - BENEFITS

What are some benefits of using Docker?

- Known/standardized configurations – the base image and any additional installed software/configurations are defined in a Dockerfile
- Can start and stop as needed. So can scale up and down as well
- Deployment platforms are built on top of Docker images (i.e., Kubernetes – K8s, OKD)

DOCKER – EXAMPLE FOR A FLASK APPLICATION

```
FROM ubuntu:22.04

LABEL maintainer="youremail@domain.tld"

RUN apt-get update -y && \
    apt-get install -y python-pip python-dev

# We copy just the requirements.txt first to leverage Docker cache
COPY ./requirements.txt /app/requirements.txt

WORKDIR /app

RUN pip install -r requirements.txt

COPY . /app


ENTRYPOINT ["python"]


CMD ["app.py"]
```

Base Image

Image Maintainer

Run Command

Copy Command

Default Dir

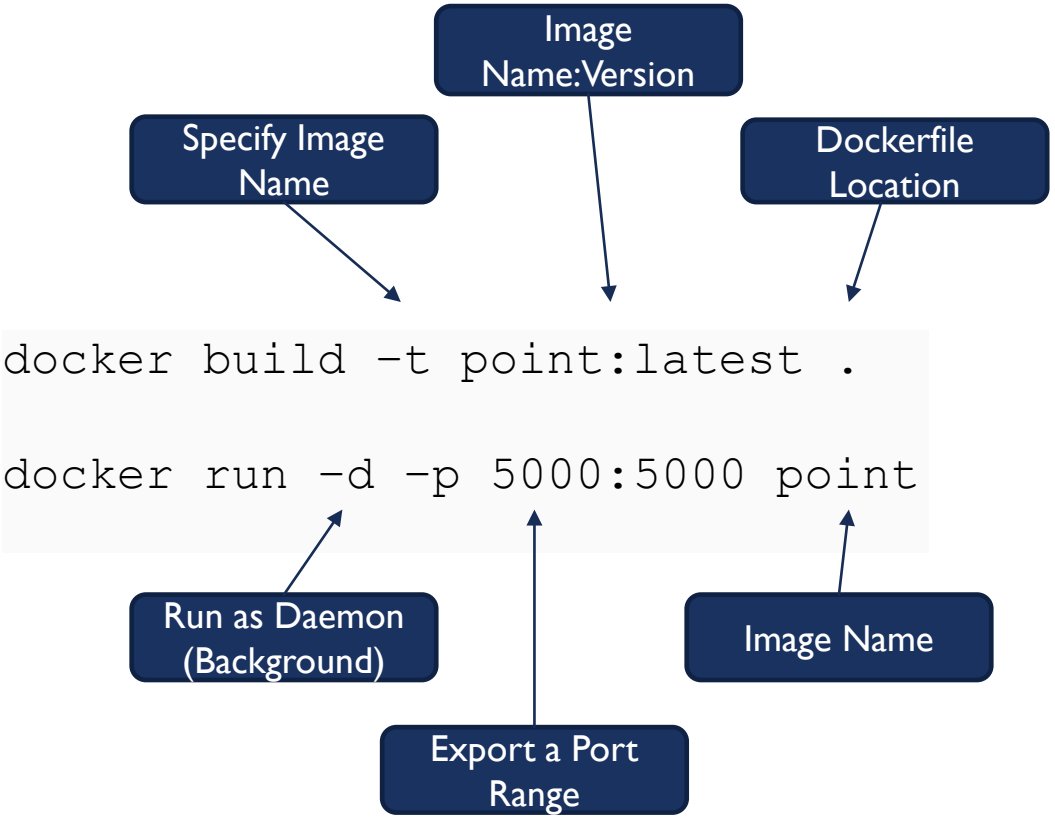
Run Command

Default Dir

Entrypoint – Run when Container Started

CMD – Argument to ENTRYPOINT

DOCKER – EXAMPLE FOR FLASK



DOCKER – RELEVANT COMMANDS

- docker build – Used to build an image from a Dockerfile
- docker images – Used to list the current set of built images
- docker run – Used to run an image as a container
- docker stop – Used to stop a running image
- docker ps – Used to list the running images

Here is a Docker cheat sheet of commands: https://devhints.io/docker

DOCKER SUMMARY

Docker	Core for building images and running containers.	Use for a small number of independent services.
Docker Compose	Build on top of Docker and part of the Docker Ecosystem.	Use for groupings of related services where the number and type are static (i.e., no scaling). Good for development and test environments, and simple production environments.
Container Orchestration	Typically 3rd party software for deployment of Docker containers, often over a cluster of VMs. Docker does have the built-in "Docker Swarm".	Use for large number of services and/or when services need to be automatically scaled up and down.

NEW LAB REQUIREMENTS

- **(Existing) REQ1180 –** The Enterprise Development Environment shall support an Artifact Management capability. This Artifact Management capability will be Sonatype Nexus 3 AND DockerHub.
- **(Existing) REQ1190 –** The Enterprise Development Environment Continuous Integration capability shall retrieve and publish artifacts from an Artifact Repository.
- **(New) REQ1200** – The Enterprise Development Environment shall use containerization to package applications for deployment to a target environment.
- **(New) REQ1230** – The Enterprise Development Environment shall hide and encrypt any passwords or secrets used in the pipeline builds.

NEW LAB REQUIREMENTS

- **(Existing) REQ1180 –** The Enterprise Development Environment shall support an Artifact Management capability. This Artifact Management capability will be Sonatype Nexus 3 AND DockerHub.

So we will use public repositories on DockerHub as our Artifact Repository for Docker images.

NEW LAB REQUIREMENTS

- **(Existing) REQ1190 –** The Enterprise Development Environment Continuous Integration capability shall retrieve and publish artifacts from an Artifact Repository.

We will publish our Docker images to repositories on DockerHub.

We will have one repository per application for the lab.

NEW LAB REQUIREMENTS

- **(New) REQ1200** – The Enterprise Development Environment shall use containerization to package applications for deployment to a target environment.

We will add a dockerfile for each repository, describing the runtime environment for the application.

We will build a Docker image as part of the build pipeline, and push it to our DockerHub repository.

Next week we will do a simple automated deployment of those containers.

RUNNING DOCKER COMMANDS

- Docker-in-Docker (DinD)
  - Runs a separate Docker Daemon in the container
- Docker-out-of-Docker (DoD)
  - Uses the Docker Daemon from the Host VM (using a Bind Mount)

We will be using the Docker-out-of-Docker method since we already have Docker installed on the Host VM

RUNNING DOCKER COMMANDS

Feature	Docker-in-Docker (DinD)	Docker-out-of-Docker (DoD)
Mechanism	Runs a separate Docker daemon inside the container.	Accesses the host's Docker daemon via socket mount.
Isolation	High isolation; inner containers are independent.	Low isolation; inner containers are siblings to outer.
Performance	Lower due to overhead of nested daemon.	Higher; leverages host resources directly.
Security	Requires --privileged mode but can be configured for isolation.	High security risk; grants root-level access to host.
Use Case	CI/CD pipelines needing isolated, consistent build environments (e.g., running untrusted code).	Scenarios prioritizing performance and ease of debugging where security is less of a concern.

SHARED LIBRARIES – PARAMETERS

- Remember that are shared libraries are Groovy functions:

python_build.groovy

```
def call() {
    <Your Pipeline Here>
}
```

Jenkinsfile

```
@Library('ci_functions@main') _
python_build()
```

SHARED LIBRARIES – PARAMETERS

- We can add parameters to those Groovy functions:

python_build.groovy

```
def call(pipeline_name) {
    ...
    sh "echo ${pipeline_name}"
    ...
}
```

Jenkinsfile

```
@Library('ci_functions@main') _

python_build('Test Pipeline')
```

SHARED LIBRARIES – CONDITIONAL STAGES

- We can make a stage run only when a condition evaluates to True:

```
stage('Package') {
    when {
        expression { env.GIT_BRANCH == 'origin/main' }
    }
    steps {
        ...
    }
}
```

Ref: https://www.jenkins.io/doc/book/pipeline/syntax/#when

We will use this in the next lab week when you exercise a Merge Request (so only pushes to Master builds and deploys a new Docker image.

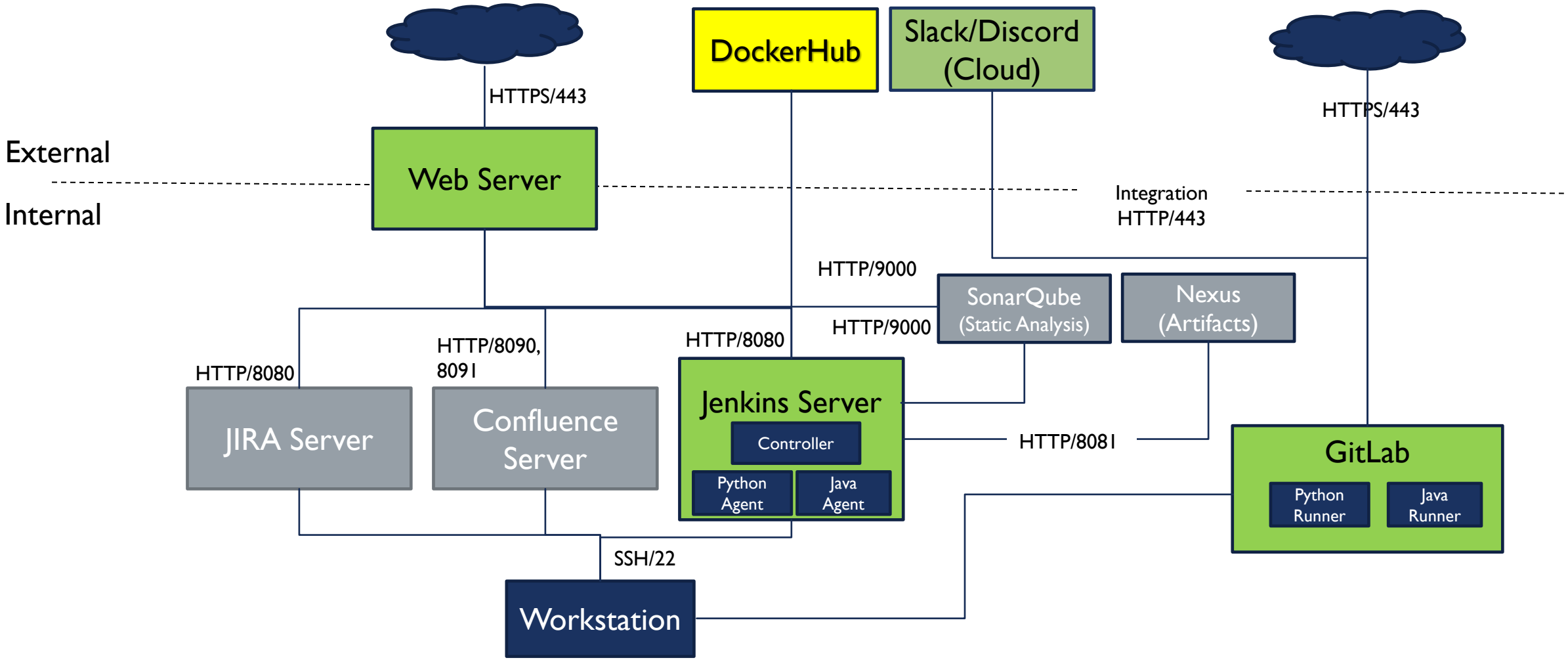
NEW LAB REQUIREMENTS

- **(New) REQ1230** – The Enterprise Development Environment shall hide and encrypt any passwords or secrets used in the pipeline builds.

We will use Jenkins credentials to store the credentials in an encrypted manner and to be able to use the credential without displaying it in the logs.

```
node {
  withCredentials([string(credentialsId: 'mytoken', variable: 'TOKEN')]) {
    sh '''
      set +x
      curl -H "Token: $TOKEN" https://some.api/
    '''
  }
}
```

YOUR ENTERPRISE DEVELOPMENT ENVIRONMENT (SO FAR)



TODAY'S LAB

Lab 10

- Posted to D2L
- Work with your partner
- Demo due by the end of next class